

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего
образования
«Омский государственный технический университет»

Факультет информационных технологий и компьютерных систем
Кафедра «Прикладная математика и фундаментальная информатика»

ОТЧЕТ
по учебной практике
(технологической (проектно-технологической) практике)

Студента Белогривцева Андрея Дмитриевича
фамилия, имя, отчество полностью

Курс 2 Группа МО-241
Направление 02.03.03 Математическое обеспечение и
администрирование информационных
систем
код, наименование

Руководитель практики от Генеральный директор ООО «Здравствуй,
мир! Технологии»
должность, название организации
организации Тюменцев Е.А.
фамилия, инициалы

18.07.2026
МП, дата, подпись руководителя практики от
организации

Руководитель доц., канд. пед. наук., доц
практики от доц.
должность, ученая степень, звание
университета Морарь Е.В.
фамилия, инициалы

дата, подпись руководителя практики от университета

Выполнил 18.07.2026
дата, подпись студента

Итоговый рейтинг	Оценка

Омск 2026

АННОТАЦИЯ

Отчёт 32 стр., 4 рис., 0 табл., 7 источн., 1 прил.

ОС АВРОРА, МОБИЛЬНАЯ РАЗРАБОТКА, РАСПОЗНАВАНИЕ РЕЧИ, БИБЛИОТЕКА VOSK, C++, QML, МНОГОПОТОЧНОСТЬ, QT MULTIMEDIA.

Объектом практики является разработка программного обеспечения для мобильной операционной системы «Аврора» в рамках выполнения кейса предприятия-партнера.

Целью технологической практики является получение профессиональных умений и опыта профессиональной деятельности в области разработки мобильных приложений с элементами машинного обучения.

Задачи практики:

- 1) расширение, систематизация и закрепление теоретических знаний по изученным дисциплинам;
- 2) сбор, обработка и анализ данных;
- 3) реализация программного обеспечения;
- 4) получение навыков командной работы.

В процессе практики были изучены архитектурные особенности и модель безопасности ОС «Аврора» (включая систему разрешений и песочницу), принципы декларативного программирования интерфейсов на QML и компоненты *Silica UI*, механизмы интеграции C++ и QML через метаобъектную систему, а также теоретические основы обработки аудиопотока (PCM, WAV) и офлайн-распознавания речи с использованием библиотеки *Vosk*.

В результате прохождения практики разработано мобильное приложение для офлайн-расшифровки речи. Реализован захват аудио с микрофона в формате PCM (16 кГц, 16 бит) средствами *Qt Multimedia*, организована многопоточная архитектура на базе *QThread* и паттерна *Worker*-объект для предотвращения блокировки графического интерфейса, выполнена интеграция *C API* библиотеки *Vosk* через прямое динамическое связывание с разделяемыми библиотеками (.so), а также реализовано корректное сохранение распознанного текста и аудиофайлов в защищённом каталоге данных приложения.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 Теория по задачам практики.....	5
1.1 Архитектура и принципы безопасности мобильной операционной системы «Аврора».....	5
1.2 Теоретические основы декларативного программирования интерфейсов на языке QML.....	6
1.3 Объектная модель фреймворка Qt и механизмы интеграции C++ с QML.....	7
1.4 Архитектура многопоточных приложений и управление платформенными ресурсами.....	8
1.5 Концепции граничных вычислений и теория инференса нейронных сетей.....	9
1.6 Теоретические аспекты кросс-компиляции и управления зависимостями в мобильных проектах.....	10
2 Разработка программного решения по заданию предприятия.....	12
2.1 Архитектура и стек технологий мобильного приложения.....	12
2.2 Интеграция моделей машинного обучения через C API Vosk.....	12
2.3 Разработка пользовательского интерфейса (QML) и логики взаимодействия.....	14
2.4 Индивидуальный вклад участника.....	15
2.5 Демонстрация работы приложения.....	16
ЗАКЛЮЧЕНИЕ.....	20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	22
ПРИЛОЖЕНИЕ А.....	23

ВВЕДЕНИЕ

С каждым годом голосовые интерфейсы становятся привычнее, но большинство решений по распознаванию речи завязаны на облачные сервисы. Это создаёт две проблемы: без интернета такие системы бесполезны, а отправка аудиозаписей на сторонние серверы вызывает вопросы с приватностью. Особенно критично это для студентов, журналистов, врачей и юристов, которые часто записывают встречи и лекции, но не готовы рисковать конфиденциальными данными.

Отечественная платформа «Аврора» даёт возможность разрабатывать полностью офлайн-приложения, используя нативные инструменты *Qt* и *C++*. Однако готовых диктофонов со встроенной расшифровкой в её экосистеме почти нет, а те, что есть, либо требуют доработки, либо не оптимизированы для работы на устройстве. Поэтому мы решили сделать собственное приложение, которое объединяет простой интерфейс, локальное распознавание речи и удобное управление заметками.

Актуальность разработки обусловлена также растущим спросом на импортозамещение программного обеспечения в корпоративном и государственном секторах. ОС «Аврора» позиционируется как безопасная мобильная платформа для работы с конфиденциальной информацией, и наличие инструментов для эффективной фиксации и обработки данных непосредственно на устройстве соответствует этой парадигме.

Технологической основой приложения стали фреймворк *Qt 5.6* с декларативным языком описания интерфейсов *QML* и компонентной библиотекой *Silica UI*, обеспечивающей нативный внешний вид в стиле ОС «Аврора».

Цель нашей работы – получение профессиональных умений и опыта профессиональной деятельности в области разработки мобильных приложений с элементами машинного обучения.

Для достижения поставленной цели были решены следующие задачи:

- 1) расширение, систематизация и закрепление теоретических знаний по изученным дисциплинам;
- 2) сбор, обработка и анализ данных;
- 3) реализация программного обеспечения;
- 4) получение навыков командной работы.

1 Теория по задачам практики

1.1 Архитектура и принципы безопасности мобильной операционной системы «Аврора»

Операционная система «Аврора» представляет собой современную мобильную платформу, разработанную компанией ООО «Открытая мобильная платформа» (ОМП) и ориентированную в первую очередь на корпоративный и государственный сегменты, где критически важны требования к информационной безопасности и технологическому суверенитету. Теоретический анализ архитектуры ОС «Аврора» показывает, что система является наследником открытых проектов *MeeGo*, *Mer* и *Sailfish OS*, что обуславливает использование ряда специфических компонентов, таких как графический стек на базе протокола *Wayland* и системный менеджер *systemd*.

Фундаментальной основой ОС «Аврора» является ядро *Linux*, которое обеспечивает низкоуровневое управление аппаратными ресурсами, процессами и памятью. Однако ключевым отличием от стандартных дистрибутивов *Linux* является реализация строгой модели изоляции приложений — «песочницы» (*sandbox*). Начиная с четвертой версии платформы, каждое пользовательское приложение запускается в изолированном окружении. Теоретически это означает, что по умолчанию процесс приложения не имеет прямого доступа к файловой системе пользователя, аппаратным датчикам и сетевым интерфейсам. Доступ к ресурсам контролируется через декларативный список разрешений (*Permissions*), который указывается в конфигурационном файле приложения формата *.desktop*. Такой подход минимизирует поверхность атаки и предотвращает утечку конфиденциальных данных, что является критически важным фактором при разработке приложений для работы с персональными и служебными данными.

Распространение программного обеспечения в экосистеме «Аврора» осуществляется посредством пакетного менеджера *RPM* (*RPM Package Manager*). Приложение упаковывается в *RPM*-пакет, который содержит не только исполняемые файлы и библиотеки, но и метаданные, файлы локализации, иконки и сценарии установки. Теория сборки *RPM*-пакетов требует понимания структуры *spec*-файла, который описывает зависимости сборки (*BuildRequires*), *runtime*-зависимости (*Requires*), а также правила размещения файлов в файловой системе устройства.

Еще одним важным теоретическим аспектом является ограничение системной версии фреймворка *Qt*. В ОС «Аврора» 5 используется *Qt* версии 5.6. Это накладывает строгие ограничения на разработчиков: в коде необходимо использовать только те импорты и API, которые были доступны в данной версии (например, `import QtQuick 2.0`), и избегать возможностей, появившихся в более поздних релизах *Qt*. Понимание этих архитектурных ограничений необходимо для создания стабильных и совместимых приложений.

1.2 Теоретические основы декларативного программирования интерфейсов на языке *QML*

Язык *QML* (*Qt Meta Language*) является декларативным языком программирования, специально разработанным для описания пользовательских интерфейсов. В отличие от императивного подхода, где разработчик последовательно описывает алгоритм изменения состояния интерфейса, в *QML* интерфейс описывается как дерево объектов с заданными свойствами и связями между ними. Теоретической основой реактивности *QML* является механизм привязок (*bindings*). Когда свойство объекта определяется через выражение, зависящее от других свойств, движок *QML* автоматически создает зависимый граф. При изменении любого из входных свойств движок пересчитывает значение зависимого свойства и обновляет визуальное представление без явного вызова методов перерисовки.

Для создания нативного пользовательского интерфейса в стиле ОС «Аврора» используется библиотека компонентов *Sailfish Silica* (в новых версиях дополняемая *Aurora Controls*). *Silica* предоставляет высокоуровневые компоненты, реализующие фирменные паттерны взаимодействия платформы: выдвижные меню (*PullDownMenu*), контекстные меню по долгому нажатию (*ContextMenu*), прокручиваемые области с инерцией (*SilicaFlickable*) и специфические элементы управления. Теория работы с *Silica* также включает понимание концепции «Обложки» (*Cover*) — интерактивного виджета, который отображается на домашнем экране в свернутом состоянии приложения и может обновляться в реальном времени.

Навигация в приложениях ОС «Аврора» строится на концепции стека страниц (*PageStack*). Каждая страница (*Page*) помещается в стек, а

переход назад осуществляется путем извлечения верхней страницы из стека. Передача данных между страницами теоретически реализуется через передачу параметров в момент вызова метода *push()*, что позволяет сохранять состояние приложения и обеспечивает предсказуемое поведение навигации. Понимание декларативной парадигмы *QML* и архитектуры *Silica* является обязательным условием для создания удобных и отзывчивых мобильных интерфейсов.

1.3 Объектная модель фреймворка *Qt* и механизмы интеграции *C++* с *QML*

Несмотря на то, что пользовательский интерфейс описывается на *QML*, вся бизнес-логика, работа с данными и производительные вычисления в приложениях ОС «Аврора» реализуются на языке *C++*. Теоретической основой взаимодействия этих двух языков является метаобъектная система *Qt* (*Meta-Object System*). Специальный компилятор метаобъектов (*МОС*) обрабатывает заголовочные файлы *C++*, содержащие макрос *Q_OBJECT*, и генерирует дополнительный код, обеспечивающий работу сигналов, слотов и системы свойств во время выполнения программы.

Механизм сигналов и слотов является фундаментальным паттерном проектирования в *Qt*, реализующим принцип слабой связанности компонентов. Сигнал — это извещение о наступлении события, а слот — это функция-обработчик. Теория соединений (*connections*) в *Qt* гарантирует, что сигналы и слоты могут быть безопасно связаны между объектами, находящимися в разных потоках выполнения, при этом *Qt* автоматически использует очередь событий (*event queue*) для потокобезопасной доставки сигналов.

Для интеграции *C++* с *QML* используются два основных теоретических подхода. Первый — публикация объекта в корневом контексте *QML* (*context property*), что делает объект глобально доступным во всех *QML*-файлах. Второй, более масштабируемый подход — регистрация типа в системе типов *QML* с помощью *qmlRegisterType*. Для того чтобы свойства *C++*-объекта могли быть использованы в привязках *QML* и автоматически обновлять интерфейс при изменении данных, они должны быть объявлены с помощью макроса *Q_PROPERTY*, где указывается геттер (*READ*) и сигнал изменения (*NOTIFY*).

Для работы со списками данных в *Qt* применяется архитектура *Model/View*. Класс *QAbstractListModel* предоставляет теоретический контракт для создания пользовательских моделей данных. Модель отвечает за хранение данных и уведомление представления (*View*) об изменениях через специальные парные вызовы методов (например, *beginInsertRows* и *endInsertRows*). Понимание объектной модели *Qt* и механизмов интеграции *C++* и *QML* позволяет создавать сложные приложения, где высокопроизводительный бэкенд на *C++* *seamlessly* взаимодействует с гибким интерфейсом на *QML*.

1.4 Архитектура многопоточных приложений и управление платформенными ресурсами

В теории разработки графических приложений существует «золотое правило»: весь пользовательский интерфейс должен работать исключительно в главном потоке (*GUI*-потоке). Главный поток отвечает за обработку событий ввода, перерисовку виджетов и обновление анимаций. Если в главном потоке выполняется длительная операция (например, чтение большого файла, сложное математическое вычисление или инференс нейронной сети), цикл обработки событий блокируется, что приводит к «зависанию» интерфейса и отсутствию отклика на действия пользователя.

Для решения этой проблемы в *Qt* реализована мощная система многопоточности. Теоретически наиболее обоснованным подходом для выполнения фоновых задач является паттерн «*Worker*-объект в *QThread*». Суть подхода заключается в создании класса-исполнителя (*Worker*), наследуемого от *QObject*, и перемещении его экземпляра в отдельный поток выполнения с помощью метода *moveToThread()*. Взаимодействие между главным потоком и потоком-исполнителем должно осуществляться исключительно через асинхронные сигналы и слоты. Это гарантирует потокобезопасность и предотвращает возникновение состояний гонки (*race conditions*) и взаимных блокировок (*deadlocks*).

Управление платформенными ресурсами в ОС «Аврора» также требует теоретического понимания работы с файловой системой в условиях песочницы. Для доступа к стандартным каталогам пользователя (Документы, Изображения, Загрузки) приложение должно запрашивать соответствующие разрешения. При этом собственные каталоги

приложения (для хранения настроек, кэша и данных) доступны без дополнительных разрешений и их пути запрашиваются через кроссплатформенный *API QStandardPaths*. Теория безопасного хранения данных диктует необходимость сериализации данных (например, в формат *JSON*) и сохранения их в изолированном каталоге приложения, что гарантирует сохранность информации даже при обновлении или переустановке приложения.

1.5 Концепции граничных вычислений и теория инференса нейронных сетей

Граничные вычисления (*Edge Computing*) — это парадигма распределенных вычислений, при которой обработка данных происходит вблизи источника их генерации (на конечном устройстве), а не в централизованном облаке. Теоретический анализ *Edge AI* (искусственного интеллекта на границе сети) выделяет ряд критических преимуществ: обеспечение конфиденциальности данных (данные не покидают устройство), возможность работы в автономном режиме (без доступа к сети), а также минимизация задержек (*latency*), так как исключается время на передачу данных по сети.

Основным форматом обмена моделями машинного обучения в индустрии является *ONNX (Open Neural Network Exchange)*. Теоретически *ONNX* представляет собой граф вычислений, где узлы — это математические операции, а ребра — многомерные массивы данных (тензоры). Для выполнения инференса *ONNX*-моделей на C++ используется библиотека *ONNX Runtime*. Теория работы с *ONNX Runtime* включает понимание жизненного цикла сессии: создание окружения (*Env*), настройка параметров сессии (*SessionOptions*), загрузка модели в память (*Session*) и непосредственный запуск вычислений (*Run*). Создание сессии является ресурсоемкой операцией, поэтому теоретически обоснованным является создание сессии один раз при запуске приложения и ее переиспользование.

Для запуска сложных моделей, таких как модель распознавания речи *GigaAM v3 RNNT (Recurrent Neural Network Transducer)*, на мобильных устройствах с ограниченными ресурсами применяется техника квантования. Теория квантования заключается в снижении точности представления весов нейронной сети с 32-битных чисел с плавающей

запятой (*float32*) до 8-битных или 16-битных целых чисел (*int8/int16*). Это позволяет в несколько раз уменьшить размер модели, занимаемый в оперативной памяти, и ускорить время инференса на *CPU* ценой незначительного снижения точности распознавания.

Архитектура приложения для офлайн-расшифровки речи теоретически должна включать конвейер обработки: запись аудиопотока с микрофона, разбиение аудио на сегменты (чанки) для поочередной обработки, передачу сегментов в *ONNX Runtime*, декодирование выходных тензоров в текст и склейку результата. Понимание теории граничных вычислений и инференса нейросетей является ключевым для создания современных интеллектуальных мобильных приложений.

1.6 Теоретические аспекты кросс-компиляции и управления зависимостями в мобильных проектах

Разработка под ОС «Аврора» ведется на хост-машине (как правило, *x86_64* под управлением *Windows* или *Linux*), в то время как целевым устройством является смартфон или планшет на базе архитектуры *ARM* (*armv7hl* или *aarch64*). Теория кросс-компиляции в данном контексте означает, что компиляция исходного кода происходит не напрямую на компьютере разработчика, а в изолированной среде сборки — *Build Engine* (виртуальная машина или *Docker*-контейнер), где установлены целевые кросс-компиляторы и системные библиотеки.

Для управления внешними зависимостями, такими как *ONNX Runtime*, которые не входят в стандартный состав ОС «Аврора», используется пакетный менеджер *Conan*. Теория работы *Conan* в проектах ОС «Аврора» основана на использовании генератора *PkgConfigDeps*, который создает файлы *.pc* (*pkg-config*). Это позволяет системе сборки *CMake* находить заголовочные файлы и библиотеки зависимостей через стандартный механизм *pkg_check_modules*.

Система сборки *CMake* предпочтительнее устаревшей *QMake* для современных проектов, так как она обеспечивает более гибкое управление зависимостями и процессом установки. Теория упаковки приложения в *RPM* требует понимания того, как динамические библиотеки (*.so*), полученные через *Conan*, должны быть деплоированы. Специальные макросы (например, *conan-deploy-libraries*) анализируют зависимости исполняемого файла и копируют необходимые разделяемые библиотеки

внутри каталога приложения (*share*/*<имя_пакета>/lib*), после чего в *spres*-файле настраиваются пути *RPATH*, чтобы динамический линковщик мог найти эти библиотеки при запуске на устройстве. Понимание теории сборки и управления зависимостями необходимо для создания переносимых и легко разворачиваемых проектов.

2 Разработка программного решения по заданию предприятия

2.1 Архитектура и стек технологий мобильного приложения

Разрабатываемое мобильное приложение «Голослов» (рабочее название) построено на нативном стеке технологий операционной системы «Аврора»: языке C++ (стандарт C++11/14) для реализации бизнес-логики и языке *QML* для описания пользовательского интерфейса. Архитектура приложения следует принципам разделения ответственности, близким к паттерну *Model-View-ViewModel (MVVM)*, где *QML*-слой отвечает исключительно за отображение данных и обработку пользовательского ввода, а классы C++ инкапсулируют работу с аппаратными ресурсами и алгоритмы распознавания.

На начальном этапе проектирования в качестве основного инструмента для инференса нейронных сетей планировалось использование библиотеки *ONNX Runtime* совместно с моделью *GigaAM*, а управление зависимостями предполагалось осуществлять через пакетный менеджер *Conan* (согласно официальным рекомендациям ООО «Открытая мобильная платформа»). Однако в ходе практической реализации данного подхода был выявлен ряд критических проблем:

1. Сложности кросс-компиляции и разрешения зависимостей *Conan* для целевых архитектур ОС «Аврора» (*armv7hl/aarch64*) в условиях ограниченных ресурсов учебной среды.
2. Чрезмерный размер бинарных файлов *ONNX Runtime* и самой модели *GigaAM*, что приводило к значительному увеличению размера *RPM*-пакета и высокому потреблению оперативной памяти на мобильных устройствах.
3. Проблемы со стабильностью сборки при интеграции *CMake* и *Conan* в единый конвейер *.spec*-файла.

В связи с этим было принято архитектурное решение отказаться от *ONNX Runtime* и *Conan* в пользу легковесной библиотеки *Vosk*. Интеграция *Vosk* была реализована через прямое динамическое связывание с предварительно скомпилированными разделяемыми библиотеками (*.so*-файлы), что кардинально упростило процесс сборки, снизило требования к памяти и обеспечило стабильную работу на эмуляторе и физических устройствах.

2.2 Интеграция моделей машинного обучения через *C API Vosk*

Для обеспечения полностью офлайн-расшифровки речи в приложение была интегрирована легковесная акустическая модель *vosk-model-small-ru-0.22*. Процесс интеграции и взаимодействия с моделью реализован с учётом строгих требований к отзывчивости пользовательского интерфейса.

Поскольку операции загрузки модели и обработки аудиопотока являются ресурсоёмкими и блокирующими, вся логика работы с *Vosk* вынесена в отдельный поток выполнения. Для этого реализован класс *VoskWorker*, наследуемый от *QObject*, который перемещается в собственный *QThread* с помощью метода *moveToThread()*. Главный класс *SpeechRecognizer* управляет жизненным циклом этого потока и обменивается с ним данными исключительно через асинхронные сигналы и слоты *Qt*, что гарантирует потокобезопасность.

При старте приложения вызывается метод *init()*, который отправляет сигнал *requestLoad* в *VoskWorker*. Воркер проверяет наличие директории с моделью и инициализирует указатели *VoskModel* и *VoskRecognizer* через функции *vosk_model_new* и *vosk_recognizer_new*.

Класс *SpeechRecognizer* настраивает *QAudioInput* на захват монофонического PCM-аудио с частотой дискретизации 16000 Гц и глубиной 16 бит (формат, требуемый моделью *Vosk*).

По мере поступления данных от микрофона (сигнал *readyRead* от *QIODevice*) байты накапливаются в буфере *m_pcm*. Для предотвращения зависания *UI* накопленные чанки данных асинхронно передаются воркеру через сигнал *requestFeed*.

Вызывается функция *vosk_recognizer_accept_waveform*.

Если функция возвращает 0, извлекается промежуточный (черновой) результат через *vosk_recognizer_partial_result*, который парсится из JSON-строки и отправляется в *QML* для отображения в реальном времени (свойство *partialText*).

Если функция возвращает 1, это означает завершение смыслового фрагмента (например, обнаружена пауза). В этом случае извлекается финальный текст через *vosk_recognizer_result* (сигнал *utterance*).

Завершение и сохранение: При вызове метода *stop()* захват аудио прекращается, и воркеру отправляется команда *requestFinalize*. Это позволяет получить последний фрагмент распознанного текста через *vosk_recognizer_final_result*.

Обработка краевых случаев: В коде реализована защита от зависания

интерфейса в случае отсутствия звука (например, при запуске на эмуляторе без настроенного микрофона). Таймер *m_noAudioTimer* отслеживает поступление данных: если в течение 4 секунд после начала записи байты не приходят, процесс корректно прерывается, ресурсы освобождаются, а пользователю выводится понятное сообщение об ошибке.

2.3 Разработка пользовательского интерфейса (QML) и логики взаимодействия

Пользовательский интерфейс приложения «Голослов» разработан с использованием декларативного языка *QML* и компонентной библиотеки *Sailfish Silica*, что обеспечивает нативный внешний вид и соответствие гайдлайнам ОС «Аврора». Архитектура взаимодействия между интерфейсом и бизнес-логикой построена на принципах слабой связанности: класс *SpeechRecognizer* зарегистрирован в *QML*-движке как синглтон (*qmlRegisterSingletonType*), что делает его глобально доступным на всех страницах приложения без необходимости ручной передачи экземпляров.

Главный экран (*MainPage.qml*) реализован на базе *SilicaListView* с кастомной темной цветовой схемой (#121212 для фона, #FFB300 для акцентов). Для управления данными используется связка *ListModel* и *JavaScript*-модуля *Database.js*, обеспечивающая фильтрацию и сортировку заметок на стороне клиента. Интерфейс поддерживает расширенные паттерны взаимодействия:

1. Выдвижное меню (*PullDownMenu*) для доступа к поиску, сортировке и информации о приложении.
2. Анимированная панель поиска с фильтрацией списка в реальном времени.

Режим множественного выбора (активируется долгим нажатием *onPressAndHold*), использующий компонент *RemorseItem* для безопасного удаления с возможностью отмены действия.

Экран записи (*RecordingPage.qml*) демонстрирует глубокую интеграцию с C++-бэкендом через элемент *Connections*. Визуальная обратная связь организована следующим образом:

Анимированный индикатор уровня звука, радиус и прозрачность которого привязаны к свойству *SpeechRecognizer.level*, вычисляемому в C++

на основе *RMS*-значений *PCM*-потока.

Таймер длительности, обновляемый через свойство *durationSec*.

Блок *live*-транскрибации, который динамически конкатенирует стабильный текст (*fullText*) и черновой гипотезу (*partialText*), поступающие от *VoskWorker* в реальном времени.

Контекстные элементы управления (старт/стоп, пауза, отмена), доступность которых (*enabled*) жестко привязана к внутреннему состоянию распознавателя (*modelReady*, *recording*, *finalizing*).

Экран просмотра заметки (*NoteViewPage.qml*) интегрирует мультимедийные возможности платформы через элемент *Audio* из модуля *QtMultimedia* 5.6. Реализован кастомный плеер с ползунком (*Slider*), привязанным к *audioPlayer.position* и *audioPlayer.duration*, что позволяет пользователю перематывать аудиозапись. Для работы с текстом предусмотрены функции копирования в буфер обмена (*Clipboard.text*) и экспорта в файл. Экспорт реализован через вызов *Q_INVOKABLE* метода *saveTextToFile* класса *SpeechRecognizer*, что гарантирует корректную запись *UTF-8* данных в защищенный каталог *Documents*. Обратная связь с пользователем при действиях (сохранение, копирование, переименование) осуществляется через систему уведомлений *Nemo.Notifications*.

Корневой файл (*goloslov.qml*) выступает в роли центрального диспетчера событий. При инициализации (*Component.onCompleted*) он запускает загрузку модели (*SpeechRecognizer.init()*). Глобальный обработчик сигнала *onFinished* перехватывает результаты записи, генерирует уникальные метаданные (дата, длительность) и инициирует сохранение записи в локальную базу данных, после чего обновляет состояние главной страницы и обложки приложения.

2.4 Индивидуальный вклад участника

Индивидуальный вклад участника заключался в проектировании и реализации пользовательского интерфейса мобильного приложения. Основным направлением работы стала разработка и адаптация *QML*-файлов с использованием компонентной библиотеки *Sailfish Silica* для обеспечения нативного внешнего вида и соответствия гайдлайнам ОС «Аврора».

В рамках этой задачи были реализованы ключевые экраны приложения: главный список заметок с функциями поиска, сортировки и

множественного выбора, экран записи с визуализацией уровня звука и отображением *live*-транскрибации, а также экран просмотра с кастомным аудиоплеером. Кроме того, была настроена корректная двусторонняя связь интерфейса с C++ бэкендом через систему сигналов и слотов Qt, реализована единая темная цветовая схема и анимации переходов для повышения удобства использования (UX).

2.5 Демонстрация работы приложения

Пользовательский интерфейс приложения разработан на декларативном языке *QML* с использованием специализированного набора компонентов *Silica*, адаптированного под жестовую среду ОС Аврора. Визуальная структура приложения разделена на четыре ключевых экрана, соответствующих логике работы с голосовыми заметками.

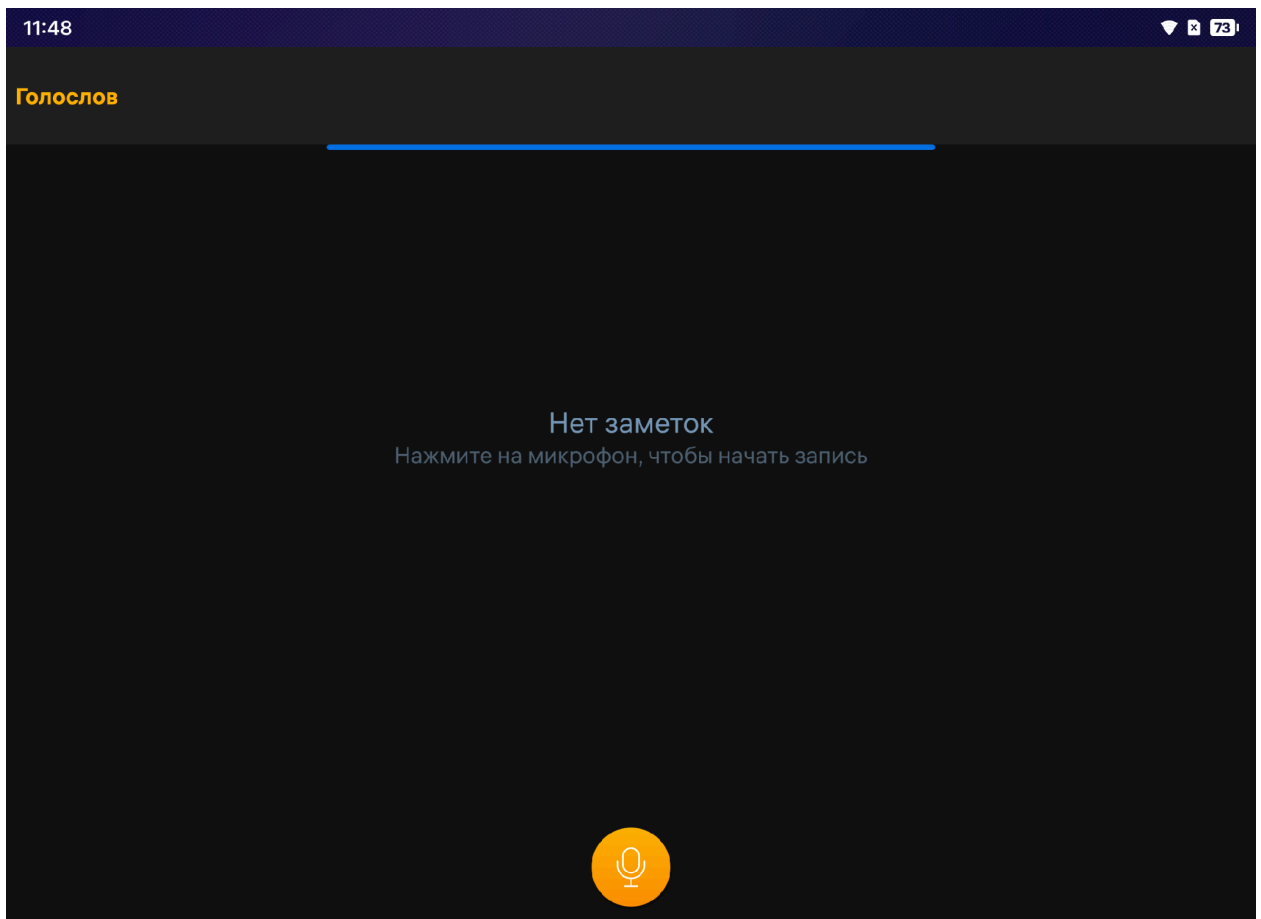


Рисунок 1 – Начальный экран

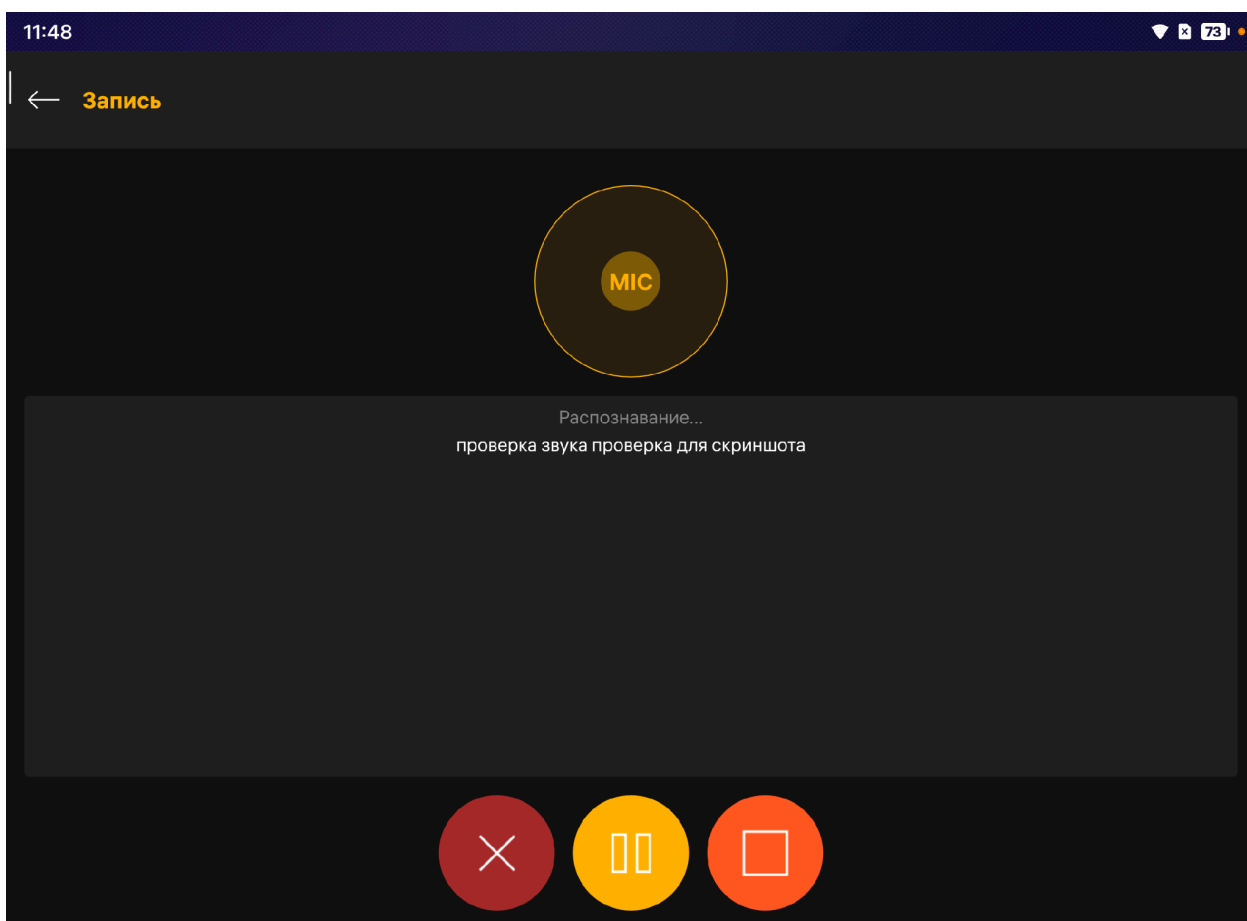


Рисунок 2 – Начало записи для расшифровки

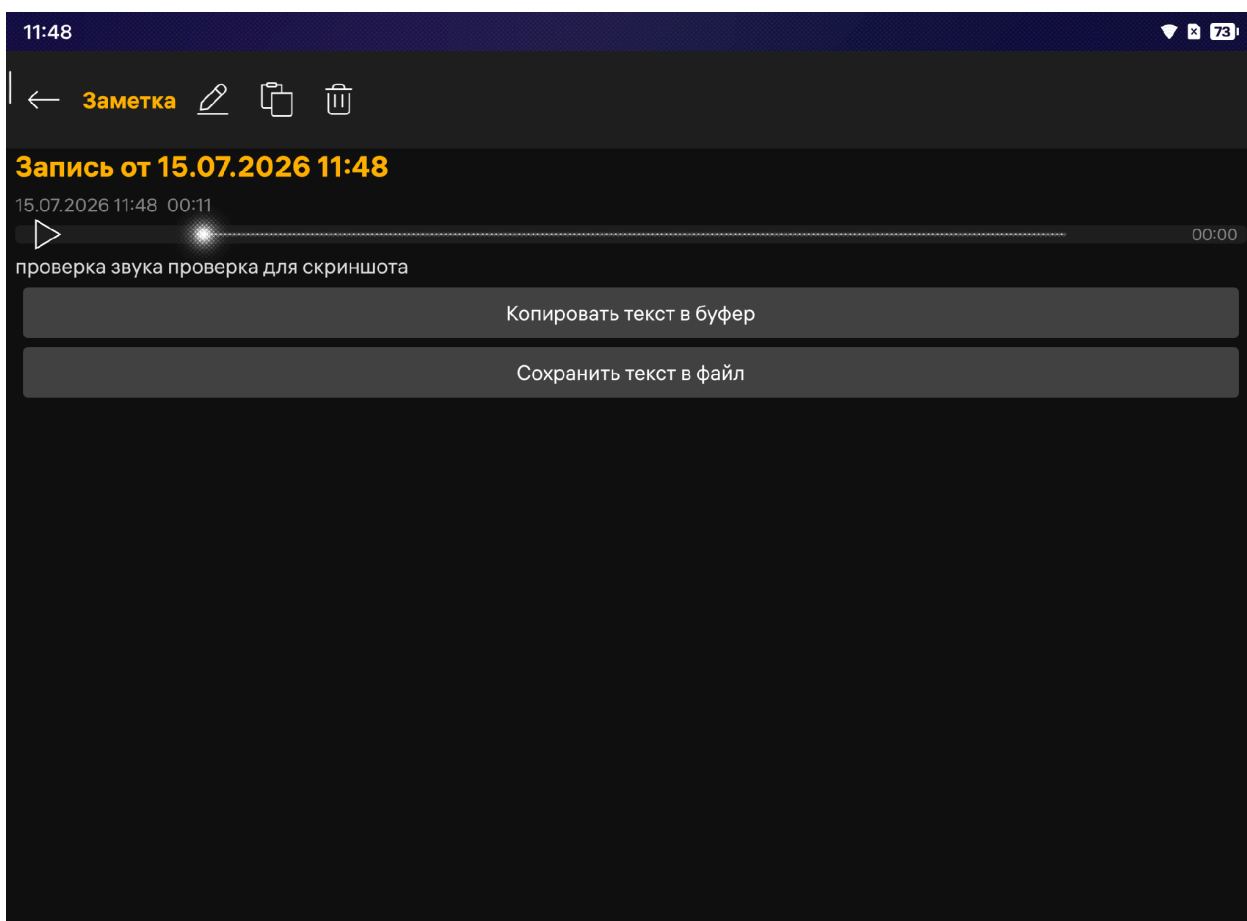


Рисунок 3 – Обработка и сохранение записи

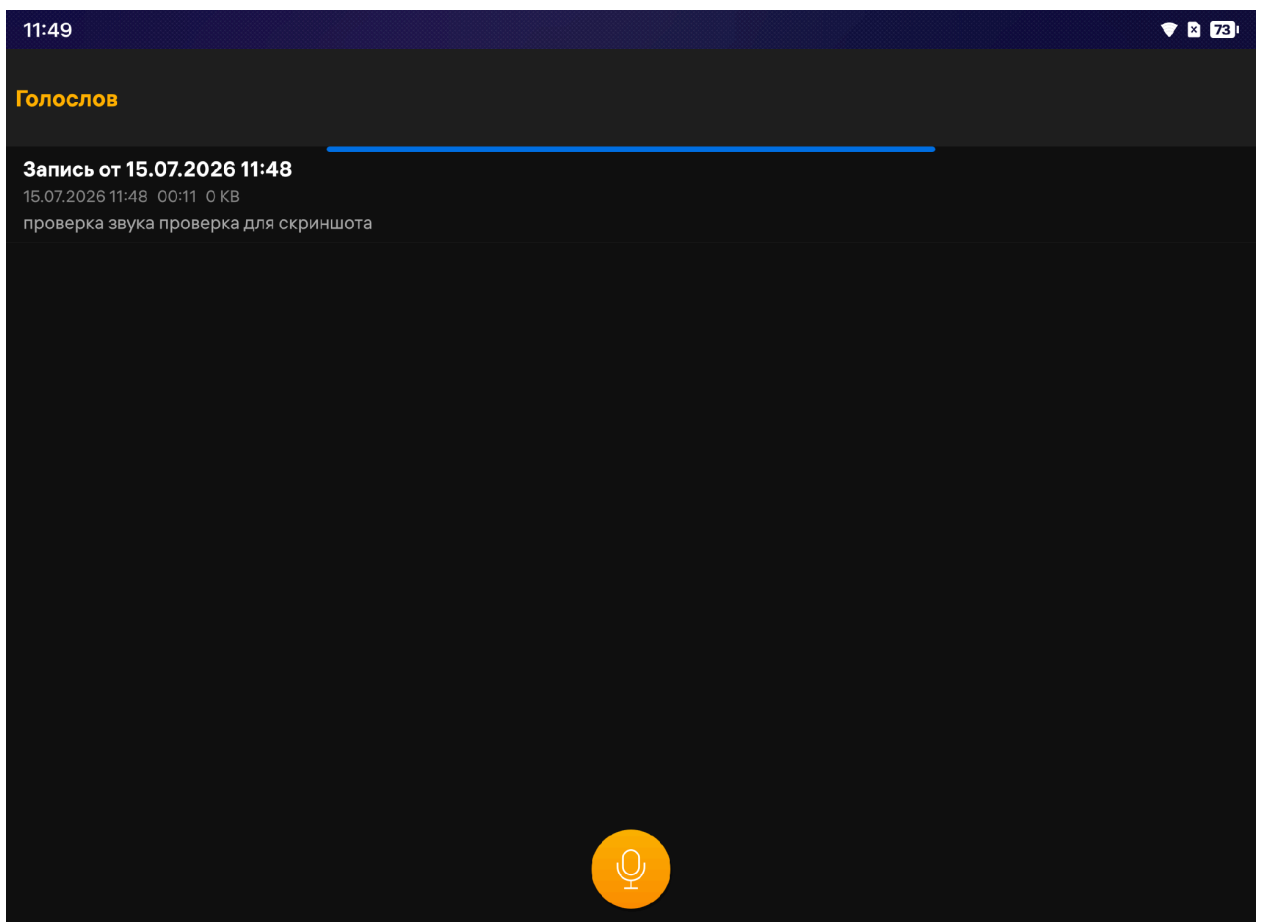


Рисунок 4 – Меню выбора существующих записей

Графический интерфейс динамически адаптируется под системную цветовую схему ОС Аврора и мгновенно реагирует на изменение состояний управляющего C++ класса *SpeechRecognizer*.

ЗАКЛЮЧЕНИЕ

Глубокое погружение в теоретические аспекты разработки под отечественные мобильные платформы стало фундаментом для всего проекта. Были детально изучены архитектура и модель безопасности ОС «Аврора», включая механизмы изоляции приложений и систему декларативных разрешений. Систематизированы знания о нативном стеке *C++/Qt/QML*, принципах построения реактивных интерфейсов на базе *Silica UI*, а также освоены концепции граничных вычислений и *C API* библиотеки *Vosk*, необходимые для запуска моделей распознавания речи непосредственно на мобильных устройствах. В ходе работы был проанализирован и обоснован переход от изначально планируемого стека *ONNX Runtime* и пакетного менеджера *Conan* к более лёгкому решению на базе *Vosk* с прямым динамическим связыванием через *.so*-библиотеки, что позволило преодолеть проблемы кросс-компиляции и существенно снизить требования к памяти устройства.

Этап сбора, обработки и анализа данных был сосредоточен на работе с аудиопотоком и структурировании пользовательской информации. Проведен анализ форматов хранения аудио (*PCM*, *WAV*) и разработаны алгоритмы разбиения длинных записей на оптимальные по размеру сегменты для последовательной подачи в модель распознавания без потери контекста. Для надежного хранения метаданных заметок и результатов транскрибации выбран формат *JSON*, что обеспечивает быструю сериализацию и целостность данных при сворачивании или аварийном завершении приложения.

Непосредственная реализация программного обеспечения включала в себя создание адаптивного пользовательского интерфейса и высокопроизводительного бэкенда. На языке *QML* спроектирован интуитивно понятный интерфейс, соответствующий гайдлайнам платформы, а на *C++* разработана многопоточная архитектура на базе *QThread* и паттерна *Worker*-объект, где фоновая расшифровка через модель *Vosk* выполняется без блокировки графического интерфейса. Особое внимание уделено корректной настройке среды сборки, интеграции внешних зависимостей через прямое динамическое связывание с разделяемыми библиотеками (*.so*) и финальной упаковке приложения в *RPM*-пакет для развертывания на эмуляторе и физических устройствах.

Процесс совместной разработки позволил приобрести и развить

навыки командной работы в условиях, приближенных к реальным индустриальным. Распределение зон ответственности между участниками — от интеграции *ML*-моделей до проектирования *UI* и модуля локального хранения — потребовало четкой координации и регулярной синхронизации. Использование систем контроля версий и совместное обсуждение архитектурных решений помогло оперативно устранять узкие места, поддерживать единый стиль кода и обеспечить слаженную работу над проектом в условиях ограниченных сроков.

Разработанное решение обладает высоким потенциалом для дальнейшего развития и практического применения. В качестве перспектив дальнейшего совершенствования продукта планируется внедрение интеллектуального разбиения текста на абзацы на основе анализа пауз, а также добавление системы тегов и папок для удобной категоризации. Кроме того, планируется реализовать функцию синхронизации текста и аудио для перехода к нужному месту записи по тапу на слове, и поддержку импорта сторонних аудиофайлов. Наиболее стабильные версии приложения планируется опубликовать в магазине *RuStore* для ОС «Аврора», сделав продукт доступным для студентов, журналистов и корпоративных пользователей, ценящих конфиденциальность данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Официальный портал разработчиков ОС Аврора. URL: <https://developer.auroraos.ru/> (дата обращения 10.07.2026)
2. Справочник компонентов Sailfish Silica и Aurora Controls. URL: <https://developer.auroraos.ru/doc/5.2.1/develop/silica/overview> (дата обращения 10.07.2026)
3. Документация и исходный код библиотеки распознавания речи Vosk. URL: <https://alphacephei.com/vosk/> (дата обращения 12.07.2026)
4. Репозиторий и документация Vosk API на GitHub. URL: <https://github.com/alphacep/vosk-api> (дата обращения 12.07.2026)
5. Официальные модели распознавания речи Vosk (включая vosk-model-small-ru-0.22). URL: <https://alphacephei.com/vosk/models> (дата обращения 12.07.2026)
6. Документация Qt 5: класс QAudioInput (захват аудиопотока). URL: <https://doc.qt.io/qt-5/qaudioinput.html> (дата обращения 13.07.2026)
7. Официальные примеры приложений с машинным обучением для ОС Аврора. URL: <https://developer.auroraos.ru/doc/5.2.1/develop/examples> (дата обращения 10.07.2026)

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг разработанного решения

Ниже представлен код файла *speechrecognizer.cpp* (логика распознавания речи):

```
#include "speechrecognizer.h"
#include "voskworker.h"

#include <QAudioDeviceInfo>
#include <QAudioFormat>
#include <QAudioInput>
#include <QDataStream>
#include <QDateTime>
#include <QDebug>
#include <QDir>
#include <QFile>
#include <QIODevice>
#include <QStandardPaths>
#include <QTextStream>
#include <QUrl>
#include <QtEndian>
#include <QtMath>

namespace {
const int kSampleRate = 16000;
const int kSampleSize = 16; // bits
const int kChannels = 1;
}

SpeechRecognizer::SpeechRecognizer(QObject *parent)
    : QObject(parent)
{
    m_modelPath(QStringLiteral("/usr/share/ru.omstu.voicenotes/models/vosk-
model-small-ru-0.22"))
        , m_sampleRate(kSampleRate)
        , m_modelReady(false)
        , m_loading(false)
        , m_recording(false)
            , m_finalizing(false)
            , m_paused(false)
            , m_level(0.0)
        , m_durationSec(0)
        , m_cancelled(false)
        , m_audioInput(nullptr)
        , m_audioIo(nullptr)
        , m_gotAudio(false)
        , m_worker(new VoskWorker)
{
}
```

```

        m_noAudioTimer.setSingleShot(true);
        m_noAudioTimer.setInterval(4000);
        connect(&m_noAudioTimer, &QTimer::timeout, this,
&SpeechRecognizer::onNoAudioTimeout);

        m_worker->moveToThread(&m_thread);
        connect(&m_thread, &QThread::finished, m_worker,
&QObject::deleteLater);

        connect(this, &SpeechRecognizer::requestLoad, m_worker,
&VoskWorker::load);
        connect(this, &SpeechRecognizer::requestFeed, m_worker,
&VoskWorker::feed);
        connect(this, &SpeechRecognizer::requestFinalize, m_worker,
&VoskWorker::finalize);
        connect(this, &SpeechRecognizer::requestReset, m_worker,
&VoskWorker::reset);

        connect(m_worker, &VoskWorker::loaded, this,
&SpeechRecognizer::onModelLoaded);
        connect(m_worker, &VoskWorker::partial, this,
&SpeechRecognizer::onPartial);
        connect(m_worker, &VoskWorker::utterance, this,
&SpeechRecognizer::onUtterance);
        connect(m_worker, &VoskWorker::finalUtterance, this,
&SpeechRecognizer::onFinalUtterance);

        m_thread.start();
    }

SpeechRecognizer::~SpeechRecognizer()
{
    teardownAudio();
    m_thread.quit();
    m_thread.wait();
}

void SpeechRecognizer::setModelPath(const QString &path)
{
    if (m_modelPath == path) {
        return;
    }
    m_modelPath = path;
    emit modelPathChanged();
}

void SpeechRecognizer::setLoading(bool v)
{
    if (m_loading != v) { m_loading = v; emit loadingChanged(); }
}

void SpeechRecognizer::setModelReady(bool v)
{
    if (m_modelReady != v) { m_modelReady = v; emit

```

```

modelReadyChanged(); }
}

void SpeechRecognizer::setRecording(bool v)
{
    if (m_recording != v) { m_recording = v; emit recordingChanged(); }
}

void SpeechRecognizer::setFinalizing(bool v)
{
    if (m_finalizing != v) { m_finalizing = v; emit finalizingChanged(); }
}

void SpeechRecognizer::setLevel(qreal v)
{
    if (!qFuzzyCompare(m_level, v)) { m_level = v; emit levelChanged(); }
}

void SpeechRecognizer::setPartialText(const QString &v)
{
    if (m_partialText != v) { m_partialText = v; emit partialTextChanged(); }
}

void SpeechRecognizer::appendText(const QString &v)
{
    const QString piece = v.trimmed();
    if (piece.isEmpty()) {
        return;
    }
    if (!m_fullText.isEmpty()) {
        m_fullText += QLatin1Char(' ');
    }
    m_fullText += piece;
    emit fullTextChanged();
}

void SpeechRecognizer::init()
{
    if (m_modelReady || m_loading) {
        return;
    }
    setLoading(true);
    emit requestLoad(m_modelPath, m_sampleRate);
}

void SpeechRecognizer::onModelLoaded(bool ok, const QString &message)
{
    setLoading(false);
    setModelReady(ok);
    if (!ok) {

```

```

        emit errorOccurred(message);
    }
}

void SpeechRecognizer::onPartial(const QString &text)
{
    if (m_recording) {
        setPartialText(text);
    }
}

void SpeechRecognizer::onUtterance(const QString &text)
{
    appendText(text);
    setPartialText(QString());
}

void SpeechRecognizer::onFinalUtterance(const QString &text)
{
    appendText(text);
    setPartialText(QString());
    setFinalizing(false);

    if (m_cancelled) {
        m_cancelled = false;
        m_pcm.clear();
        return;
    }

    QString audioUrl;
    if (!m_pcm.isEmpty()) {
        const QString path = writeWav(m_pcm);
        if (!path.isEmpty()) {
            audioUrl = QUrl::fromLocalFile(path).toString();
        }
    }
    const int dur = m_durationSec;
    m_pcm.clear();
    emit finished(m_fullText, audioUrl, dur);
}

void SpeechRecognizer::start()
{
    if (m_recording || !m_modelReady) {
        return;
    }

    m_cancelled = false;
    m_pcm.clear();
    m_fullText.clear();
    emit fullTextChanged();
    setPartialText(QString());
    m_durationSec = 0;
    emit durationSecChanged();
}

```

```

emit requestReset();

QAudioFormat format;
format.setSampleRate(m_sampleRate);
format.setChannelCount(kChannels);
format.setSampleSize(kSampleSize);
format.setCodec(QStringLiteral("audio/pcm"));
format.setByteOrder(QAudioFormat::LittleEndian);
format.setSampleType(QAudioFormat::SignedInt);

QAudioDeviceInfo info = QAudioDeviceInfo::defaultInputDevice();
if (info.isNull()) {
    emit errorOccurred(tr("Микрофон недоступен"));
    return;
}
if (!info.isFormatSupported(format)) {
    format = info.nearestFormat(format);
}

m_audioInput = new QAudioInput(info, format, this);
m_audioIo = m_audioInput->start();
if (!m_audioIo) {
    teardownAudio();
    emit errorOccurred(tr("Не удалось начать запись с
микрофона"));
    return;
}
connect(m_audioIo, &QIODevice::readyRead, this,
&SpeechRecognizer::onAudioDataReady);
setLevel(0.0);
setRecording(true);
m_gotAudio = false;
m_noAudioTimer.start();
}

void SpeechRecognizer::onAudioDataReady()
{
    if (!m_audioIo) {
        return;
    }
    const QByteArray data = m_audioIo->readAll();
    if (data.isEmpty()) {
        return;
    }
    m_gotAudio = true;
    m_noAudioTimer.stop();
    m_pcm.append(data);

    // Compute an RMS level for the meter and update duration.
    const int sampleCount = data.size() / 2;
    if (sampleCount > 0) {
        const qint16 *samples = reinterpret_cast<const qint16
*>(data.constData());
        qreal sumSquares = 0.0;

```

```

        for (int i = 0; i < sampleCount; ++i) {
            const qreal s = static_cast<qreal>(samples[i]) / 32768.0;
            sumSquares += s * s;
        }
        const qreal rms = qSqrt(sumSquares / sampleCount);
        // Scale up a little so normal speech fills the meter.
        setLevel(qBound<qreal>(0.0, rms * 3.0, 1.0));
    }

    const int totalSamples = m_pcm.size() / 2;
    const int dur = totalSamples / m_sampleRate;
    if (dur != m_durationSec) {
        m_durationSec = dur;
        emit durationSecChanged();
    }

    emit requestFeed(data);
}

void SpeechRecognizer::onNoAudioTimeout()
{
    if (!m_recording || m_gotAudio) {
        return;
    }
    m_cancelled = true;
    teardownAudio();
    setRecording(false);
    setLevel(0.0);
    setPartialText(QString());
    m_pcm.clear();
    m_fullText.clear();
    emit fullTextChanged();
    emit requestReset();
    emit errorOccurred(tr("Микрофон не передаёт звук. В эмуляторе  
запись микрофона "
                           "обычно недоступна – проверьте на устройстве  
или включите "
                           "аудиовход эмулятора."));
}

void SpeechRecognizer::stop()
{
    if (!m_recording) {
        return;
    }
    m_noAudioTimer.stop();
    setPaused(false);
    if (m_audioIo) {
        const QByteArray tail = m_audioIo->readAll();
        if (!tail.isEmpty()) {
            m_pcm.append(tail);
            emit requestFeed(tail);
        }
    }
}

```

```

        teardownAudio();
        setRecording(false);
        setLevel(0.0);
        setFinalizing(true);
        emit requestFinalize();
    }

void SpeechRecognizer::cancel()
{
    if (!m_recording && !m_finalizing) {
        return;
    }
    m_cancelled = true;
    m_noAudioTimer.stop();
    setPaused(false);
    teardownAudio();
    setRecording(false);
    setLevel(0.0);
    setPartialText(QString());
    m_pcm.clear();
    m_fullText.clear();
    emit fullTextChanged();
    if (m_finalizing) {
        return;
    }
    setFinalizing(true);
    emit requestFinalize();
}

void SpeechRecognizer::pause()
{
    if (!m_recording || m_paused)
        return;

    teardownAudio();
    setPaused(true);
}

void SpeechRecognizer::resume()
{
    if (!m_recording || !m_paused)
        return;

    QAudioFormat fmt;
    fmt.setSampleRate(m_sampleRate);
    fmt.setChannelCount(1);
    fmt.setSampleSize(16);
    fmt.setCodec("audio/pcm");
    fmt.setByteOrder(QAudioFormat::LittleEndian);
    fmt.setSampleType(QAudioFormat::SignedInt);

    QAudioDeviceInfo info = QAudioDeviceInfo::defaultInputDevice();
    if (!info.isFormatSupported(fmt)) {
        emit errorOccurred(tr("Микрофон недоступен"));
    }
}

```

```

        return;
    }

    m_audioInput = new QAudioInput(info, fmt);
    m_audioIo = m_audioInput->start();
    if (!m_audioIo) {
        emit errorOccurred(tr("Не удалось начать запись с
микрофона"));
        return;
    }

    connect(m_audioIo, &QIODevice::readyRead, this,
&SpeechRecognizer::onAudioDataReady);
    setPaused(false);
}

void SpeechRecognizer::setPaused(bool v)
{
    if (m_paused != v) {
        m_paused = v;
        emit pausedChanged();
    }
}

void SpeechRecognizer::teardownAudio()
{
    if (m_audioIo) {
        disconnect(m_audioIo, &QIODevice::readyRead, this,
&SpeechRecognizer::onAudioDataReady);
        m_audioIo = nullptr;
    }
    if (m_audioInput) {
        m_audioInput->stop();
        m_audioInput->deleteLater();
        m_audioInput = nullptr;
    }
}

QString SpeechRecognizer::writeWav(const QByteArray &pcm) const
{
    QString dirPath =
QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
    if (dirPath.isEmpty()) {
        dirPath = QDir::homePath() +
QStringLiteral("/.local/share/ru.omstu.voicenotes");
    }
    dirPath += QStringLiteral("/audio");
    QDir dir;
    if (!dir.mkpath(dirPath)) {
        return QString();
    }

    const QString fileName = QStringLiteral("note_%1.wav")

```

```

.arg(QDateTime::currentDateTime().toString(QStringLiteral("yyyyMMdd_hh
mmss"))));
    const QString fullPath = dirPath + QLatin1Char('/') + fileName;

    QFile file(fullPath);
    if (!file.open(QIODevice::WriteOnly)) {
        return QString();
    }

    const quint32 dataSize = static_cast<quint32>(pcm.size());
    const quint16 channels = kChannels;
    const quint32 sampleRate = static_cast<quint32>(m_sampleRate);
    const quint16 bitsPerSample = kSampleSize;
    const quint32 byteRate = sampleRate * channels * (bitsPerSample /
8);
    const quint16 blockAlign = channels * (bitsPerSample / 8);
    const quint32 riffSize = 36 + dataSize;

    QDataStream out(&file);
    out.setByteOrder(QDataStream::LittleEndian);

    file.write("RIFF", 4);
    out << riffSize;
    file.write("WAVE", 4);
    file.write("fmt ", 4);
    out << quint32(16);
    out << quint16(1);
    out << channels;
    out << sampleRate;
    out << byteRate;
    out << blockAlign;
    out << bitsPerSample;
    file.write("data", 4);
    out << dataSize;
    file.write(pcm);
    file.close();

    return fullPath;
}

bool SpeechRecognizer::saveTextToFile(const QString &filePath, const
QString &text)
{
    QString localPath = filePath;
    // Handle file:// URLs
    if (localPath.startsWith(QLatin1String("file://"))) {
        QUrl url(localPath);
        localPath = url.toLocalFile();
    }

    QFile file(localPath);
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
        return false;
    }
}

```

```

        QTextStream stream(&file);
        stream.setCodec("UTF-8");
        stream << text;
        file.close();
        return true;
    }

int SpeechRecognizer::fileSize(const QString &path) const
{
    QString localPath = path;
    if (localPath.startsWith(QLatin1String("file://"))) {
        QUrl url(localPath);
        localPath = url.toLocalFile();
    }
    QFile f(localPath);
    if (!f.exists()) {
        return 0;
    }
    return static_cast<int>(f.size());
}

```